

CPS209 Lab 7

Swing & 2D Shapes

Preamble

In this lab you will practice creating a Swing interface that includes 2D shapes. Your interface will react to mouse movement using a `MouseListener`.

Homework to Complete (Submit on D2L):

Animating Four-Heads

Fill in the `Head` class in `Head.java` according to the description below. There is no automatic tester for this one since the proof is in the visualization. Run the main function in `HeadMain.java` to verify that your graphics and Swing components are being rendered and behaving correctly.

Create a custom Swing GUI component (started for you in `Head.java`) that extends `JPanel`. This component should display a simple human head that reacts to the mouse cursor entering and exiting the surface of that component. At minimum, your `Head` class will require the following methods:

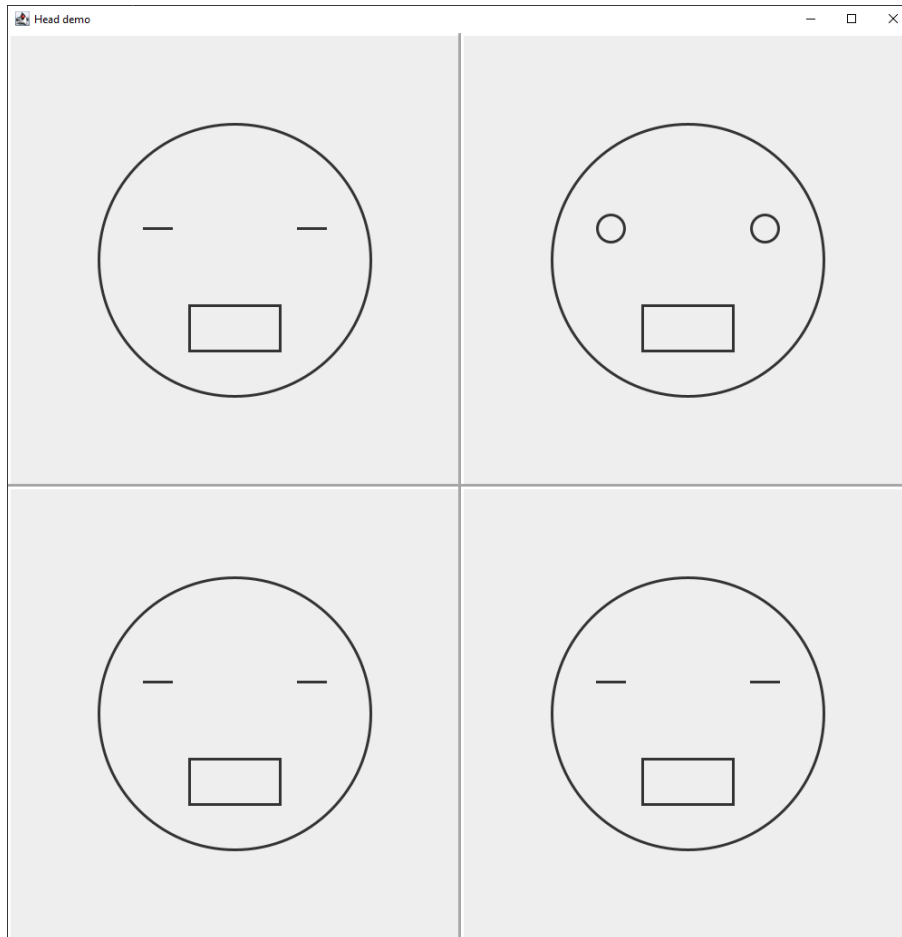
`public Head()`

The constructor of the class should first set the preferred size of this component to be 500-by-500 pixels. Next, this constructor should create and add a `MouseListener` to this component. Continue reading to understand what mouse events we want to react to.

`@Override public void paintComponent(Graphics g)`

This method renders some kind of abstract cartoon image of a simple human head on the surface of this component. If the mouse cursor is hovering over the head, the eyes should be drawn to look open. If the mouse cursor is outside the head, the eyes should be drawn to look closed. This is not an art class, so this head does not need to be pretty. A couple of simple rectangles and ovals will suffice. Of course, students with a more artistic bent and interest might want to check out more complicated shapes from the package [java.awt.geom](#) to render a prettier image.

To admire your interactive Swing component, you can run the main method in the provided `HeadMain` class. This main method creates a `JFrame` that contains four separate `Head` components arranged in a neat 2-by-2 grid using the `GridLayout` layout manager, as in the screenshot below. Then you can wiggle your mouse cursor from the top of one component onto another to watch the eyes open and close with your mouse movements.



Reminder

- Your methods should be SILENT. Print statements are fine for testing and debugging but remove them from your code before you submit.
- No method should take longer than a couple of seconds to pass the tester. Methods that take too long will be rejected. Writing efficient code is an important part of this course.

Submission

Labs are to be completed and submitted **individually**. Submit your completed **Head.java** on D2L, under the submission for Lab 7.

Credit where credit is due:

This problem is adapted from Ilkka Kokkarinen's excellent "CCPS 209 Computer Science II Labs" problem set. Credit for this problem goes to him, and the full set of 99 can be found at his Github page: <https://github.com/ikokkari>